# REPLIC

## Reinforcement-Learning-Engineered Protocol for Internet Congestion

A reliable transport over UDP in C with a learned (PPO) congestion controller

**Team:** Harshavardhan Mallela (24BAI0274), Rohit Arya (24BCE0819), Shloak Sinha (24BDS0378)

**Course:** Computer Networks, VIT Vellore **Faculty:** Anuradha G. **Date:** 21 October 2026

**Abstract.** UDP is fast but unreliable, and TCP lives inside the kernel where it cannot be changed. REPLIC rebuilds reliable delivery on UDP in user space: a 13-byte header with CRC-16, sequence numbers, Selective Repeat with SACK, Jacobson-style timeouts and optional XOR forward error correction. Its congestion controller is a small neural network (8-32-32-1) trained with Proximal Policy Optimization, with a fairness term in the reward. On 30 simulated cellular-like paths it moves 1 MB in 6.1 s against 14.9 s for the original fixed-window AIMD sender, and it shares a link fairly with CUBIC (Jain index 0.97).

**Key facts.** Payload up to 1200 B. Window 256 packets. Controller: cwnd = cwnd x 2^a, a in (-1, 1). Baselines: AIMD, CUBIC, BBR-style, DQN agent.

Fig 1. REPLIC architecture

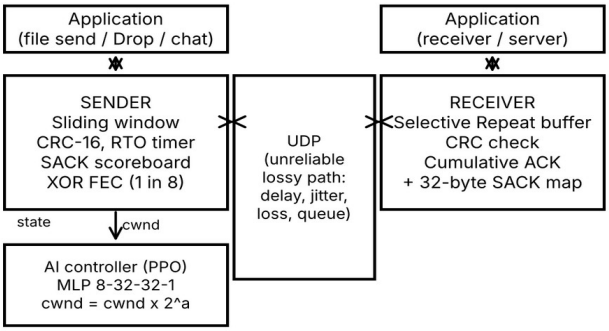
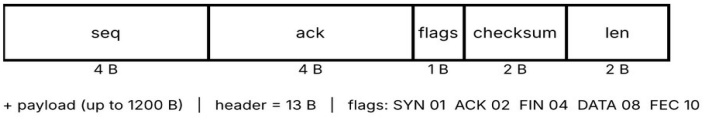
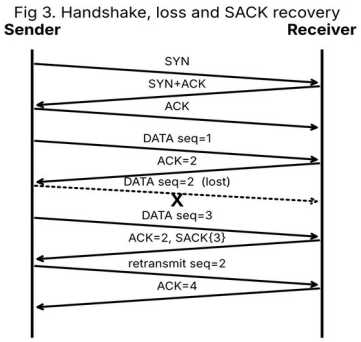



Fig 2. REPLIC packet header




+ payload (up to 1200 B) | header = 13 B | flags: SYN 01 ACK 02 FIN 04 DATA 08 FEC 10

Fig 3. Handshake, loss and SACK recovery



**Code 1 of 3: packet format and CRC-16 (src/common.h, src/common.c)**

```
#define FLAG_SYN  0x01
#define FLAG_ACK  0x02
#define FLAG_FIN  0x04
#define FLAG_DATA 0x08
#define FLAG_FEC  0x10

typedef struct {          /* header 13 B */
    uint32_t seq_num;
    uint32_t ack_num;
    uint8_t  flags;
    uint16_t checksum;
    uint16_t payload_len;
    uint8_t  payload[1200];
} replic_packet_t;

static uint16_t crc16_byte(uint16_t crc, uint8_t b) {
    crc ^= (uint16_t)b << 8;
    for (int k = 0; k < 8; k++)
        crc = (crc & 0x8000)
            ? (uint16_t)((crc << 1) ^ 0x1021)
            : (uint16_t)(crc << 1);
    return crc;
}

uint16_t replic_checksum(const replic_packet_t *p) {
    uint16_t crc = 0xFFFF;
    for (int s = 24; s >= 0; s -= 8) crc = crc16_byte(crc, p->seq_num >> s);
    for (int s = 24; s >= 0; s -= 8) crc = crc16_byte(crc, p->ack_num >> s);
    crc = crc16_byte(crc, p->flags);
    crc = crc16_byte(crc, p->payload_len >> 8);
    crc = crc16_byte(crc, p->payload_len);
    for (int i = 0; i < p->payload_len; i++)
        crc = crc16_byte(crc, p->payload[i]);
    return crc;      /* checksum field itself is excluded */
}
```

**Code 2 of 3: Selective Repeat receiver and SACK acknowledgement  (src/server.c)**

```
/* SRW = 256 packets. have[] marks buffered packets. */

on_data(seq, payload):
    if seq == expected:                 /* in order */
        deliver(payload)
        expected++
        while have[expected % SRW]:     /* drain buffer */
            deliver(buf[expected % SRW])
            have[expected % SRW] = 0
            expected++
    else if seq > expected and seq - expected < SRW:
        buf[seq % SRW]  = payload       /* out of order */
        have[seq % SRW] = 1
    else:
        ignore                          /* duplicate */

send_ack():
    ack.ack_num     = expected          /* cumulative */
    ack.flags       = FLAG_ACK
    ack.payload_len = SRW / 8           /* 32-byte SACK map */
    for j = 1 to SRW - 1:
        if have[(expected + j) % SRW]:
            ack.payload[j / 8] |= 1 << (j % 8)
    send(ack)

/* bit j of the map = packet (ack + j) has arrived;
   bit 0 is unused because packet ack is the missing one */
```

### Code 3 of 3: AI controller, timeout, loss detection (src/smart_cc.h, src/replic_sender.c)

```
/* AI controller: 8 inputs -> 32 -> 32 -> 1 */
sm_forward(x[8]):
    h1 = tanh(W1 * x  + b1)             /* 32 units */
    h2 = tanh(W2 * h1 + b2)             /* 32 units */
    o  = W3 * h2 + b3
    return tanh(o)                      /* a in (-1, 1) */

/* each control interval */
a    = sm_forward(state)
cwnd = cwnd * 2^a                       /* x0.5 to x2 */
cwnd = clamp(cwnd, 2, maxcwnd)

/* retransmission timer: Jacobson + Karn */
rttvar = 0.75 * rttvar + 0.25 * |srtt - rtt|
srtt   = 0.875 * srtt  + 0.125 * rtt
rto    = srtt + 4 * rttvar              /* clamp 60..3000 ms */
/* Karn: never sample RTT from a retransmitted packet */

/* SACK loss detection */
cnt = 0
for s = next - 1 down to base:
    if sacked[s]:  cnt++
    else if cnt >= 3:  retransmit(s)    /* 3 later packets seen */

/* XOR FEC: after every 8 DATA packets send parity
   p1 ^ p2 ^ ... ^ p8; one loss per group is rebuilt
   without a retransmission. */

/* state: RTT gradient, queue-delay trend, queue-delay level,
   loss ratio, inflight/BDP, competing-traffic pressure,
   previous action, log2(cwnd)/7 */
```

```
Complete source code, tests and lab files:  https://github.com/Capsturrrr/rudp
```